NOSQL PROJECT · NETFLIX DATASET

PostgreSQL vs Neo4j

Comparing a relational DBMS and a graph database in terms of efficiency

Andrei Rares Grigore (1953878)

The Dataset

8,807
titles · Kaggle, Netflix Movies and TV Shows

12
columns in one denormalized table

ATTRIBUTES OF A ROW

show_id	type	title	**director**	**cast**	**country**
date_added	release_year	rating	duration	**listed_in**	description

Multi-valued — several comma-separated values in one cell

Data Cleaning

`clean_netflix_data.py`

a **Split multi-valued cells**

Each comma-separated value becomes one row of a bridge table: one cast cell → several (show_id, person) rows.

b **Convert dates**

`"September 25, 2021"` → `2021-09-25`, storable as `DATE`.

c **Remove duplicates**

Same actor listed twice in one cell. `acted_in 64,126 → 64,124` · `directed 6,978 → 6,977`. Prevents a PostgreSQL key error and duplicate Neo4j edges.

d **Write two CSV formats**

`POSTGRESQL · NUMERIC IDS`

```
show_id,person_id
s2,1829
s2,20390
```

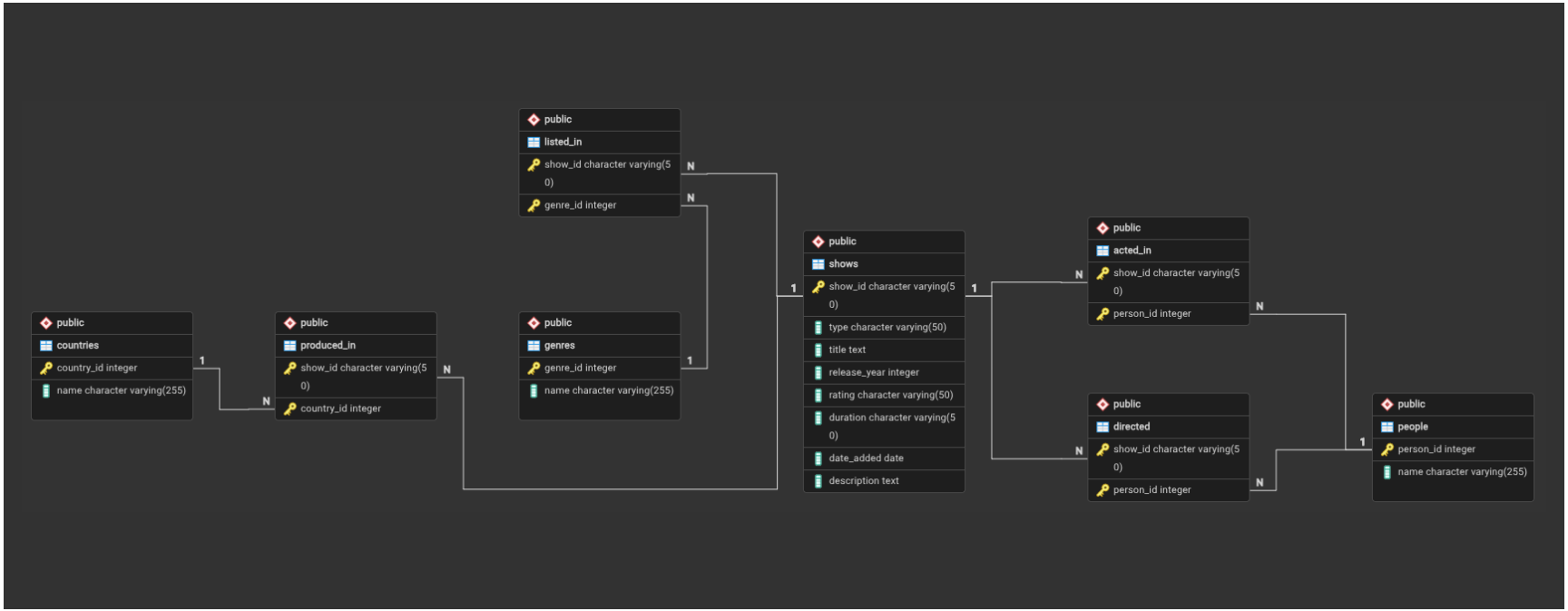
`NEO4J · NAMES`

```
show_id,person_name
s2,Ama Qamata
s2,Khosi Ngema
```

Foreign keys need IDs; Neo4j matches both nodes by name to create an edge.

The Relational Model

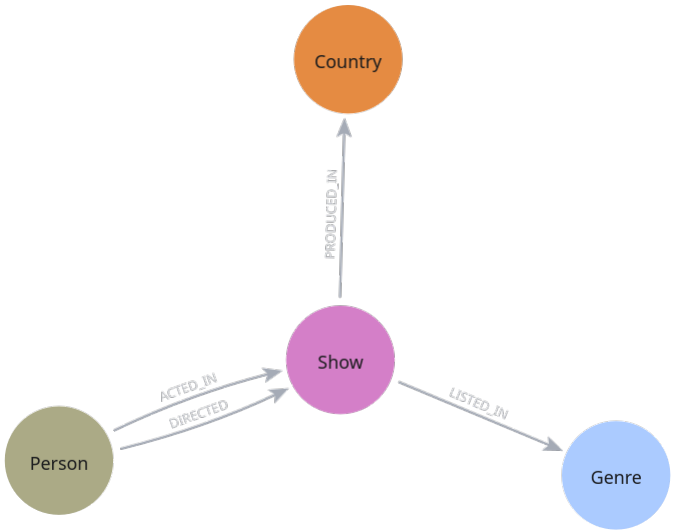
POSTGRESQL · 3NF · 4 ENTITY TABLES + 4 BRIDGE TABLES



Bridge tables and foreign keys: every hop is a join.

The Graph Model

NEO4J · PROPERTY GRAPH · 4 NODE LABELS + 4 RELATIONSHIP TYPES



Direct edges: every hop follows a pointer (index-free adjacency).

The Queries

Ordered from simple lookups to deep traversals, each described by the number of graph edges it crosses.

QUERY	WHAT IT ASKS	EDGES TRAVERSED
Q1	Shows of a genre	1
Q2	Cast of a show	1
Q3	Directors who also acted in their own title	2
Q4	Actors reachable in exactly 3 co-acting hops	6
Q5a–Q5e	Degrees of separation at real distance 1 to 5	2, 4, 6, 8, 10

Q1 — Shows by Genre

All titles in the genre “Documentaries”.

1 edge 869 rows

SQL

```
SELECT s.title, s.release_year FROM shows s
JOIN listed_in li ON s.show_id = li.show_id
JOIN genres g ON li.genre_id = g.genre_id
WHERE g.name = 'Documentaries';
```

CYPHER

```
MATCH (s:Show)-[:LISTED_IN]->(g:Genre {name: 'Documentaries'})
RETURN s.title, s.release_year;
```

SQL rebuilds the link through three tables; Cypher starts from the Genre node and follows its incoming edges.

WARM MEDIAN PostgreSQL 2.4 ms Neo4j 28.1 ms **PostgreSQL 12×**

Q2 — Cast of a Show

The actors of “Breaking Bad”.

1 edge 10 rows

SQL

```
SELECT p.name FROM acted_in ai
JOIN people p ON ai.person_id = p.person_id
JOIN shows s ON ai.show_id = s.show_id
WHERE s.title = 'Breaking Bad';
```

CYPHER

```
MATCH (p:Person)-[:ACTED_IN]->(s:Show {title: 'Breaking Bad'})
RETURN p.name;
```

Both find the show through a title index, then its actors: via the acted_in key in SQL, via incoming edges in Cypher.

WARM MEDIAN PostgreSQL 0.3 ms Neo4j 6.5 ms **PostgreSQL 22×**

Q2 — Cast of a Show

The show node with everything connected to it: its cast, its genres and its country.



Q3 — Directors Who Also Acted

People who both directed and acted in the same title.

2 edges 355 rows

SQL

```
SELECT DISTINCT p.name, s.title FROM directed d
JOIN acted_in a ON d.show_id = a.show_id AND d.person_id = a.person_id
JOIN people p ON d.person_id = p.person_id
JOIN shows s ON d.show_id = s.show_id;
```

CYPHER

```
MATCH (p:Person)-[:DIRECTED]->(s:Show)<-[:ACTED_IN]-(p)
RETURN DISTINCT p.name, s.title;
```

“Same show and same person”: a join on both columns in SQL, the variable p reused at both ends in Cypher.

WARM MEDIAN PostgreSQL 12.2 ms Neo4j 47.1 ms **PostgreSQL 3.9×**

Q4 — Actors at 3 Hops

Every actor reachable from Leonardo DiCaprio in exactly three co-acting links.

6 edges 13,326 rows

SQL · SIX JOINS ON ACTED_IN

```
SELECT DISTINCT p3.name FROM people p1
JOIN acted_in ai1 ON p1.person_id = ai1.person_id
JOIN acted_in ai2 ON ai1.show_id = ai2.show_id AND ai1.person_id != ai2.person_id
JOIN acted_in ai3 ON ai2.person_id = ai3.person_id AND ai3.show_id != ai1.show_id
JOIN acted_in ai4 ON ai3.show_id = ai4.show_id AND ai4.person_id != ai3.person_id
JOIN acted_in ai5 ON ai4.person_id = ai5.person_id AND ai5.show_id != ai3.show_id
JOIN acted_in ai6 ON ai5.show_id = ai6.show_id AND ai6.person_id != ai5.person_id
JOIN people p3 ON ai6.person_id = p3.person_id
WHERE p1.name = 'Leonardo DiCaprio' AND p3.person_id != p1.person_id;
```

CYPHER

```
MATCH (p1:Person {name: 'Leonardo DiCaprio'})-[:ACTED_IN*6]-(p3:Person)
WHERE p1 <> p3 RETURN DISTINCT p3.name;
```

WARM MEDIAN PostgreSQL 118.2 ms Neo4j 215.9 ms **PostgreSQL 1.8×**

Q5 – Degrees of Separation

Minimum number of co-acting links between two actors, at real distance 1 to 5 (Q5a–Q5e). Shown: Q5c. 2–10 edges

CYPHER · SHORTESTPATH()

```
MATCH path = shortestPath(
    (p1:Person {name: 'Ricky Gervais'})
    -[:ACTED_IN*..6]-
    (p2:Person {name: 'Rowan Atkinson'})
)
RETURN length(path) AS degrees_of_separation;
```

SQL · BREADTH-FIRST CTE

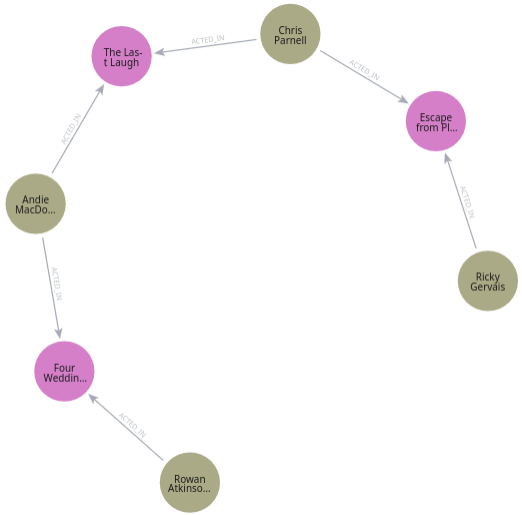
```
WITH RECURSIVE bfs(person_id, depth) AS (
    SELECT person_id, 0 FROM people WHERE name = 'Ricky Gervais'
    UNION
    SELECT ai2.person_id, b.depth + 1
    FROM bfs b
    JOIN acted_in ai1 ON ai1.person_id = b.person_id
    JOIN acted_in ai2 ON ai2.show_id = ai1.show_id
    WHERE b.depth < 3
)
SELECT MIN(depth) AS degrees_of_separation FROM bfs
JOIN people p ON p.person_id = bfs.person_id
WHERE p.name = 'Rowan Atkinson';
```

Cypher counts edges, SQL counts people: the Cypher limit is always twice the SQL one (*..6 vs depth < 3).

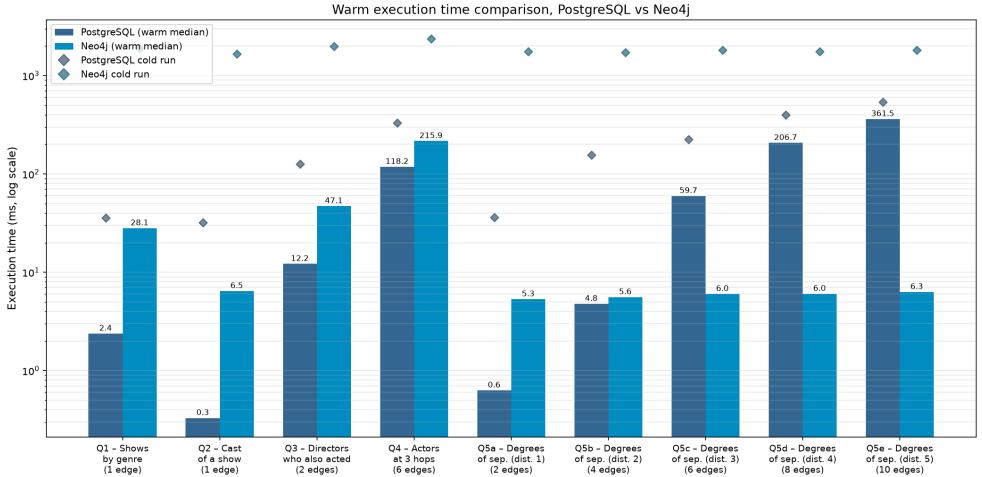
WARM MEDIAN PostgreSQL 59.7 ms Neo4j 6.0 ms **Neo4j 10×**

Q5 — A Shortest Path

Ricky Gervais to Rowan Atkinson: three co-acting hops, six `ACTED_IN` edges.



Results

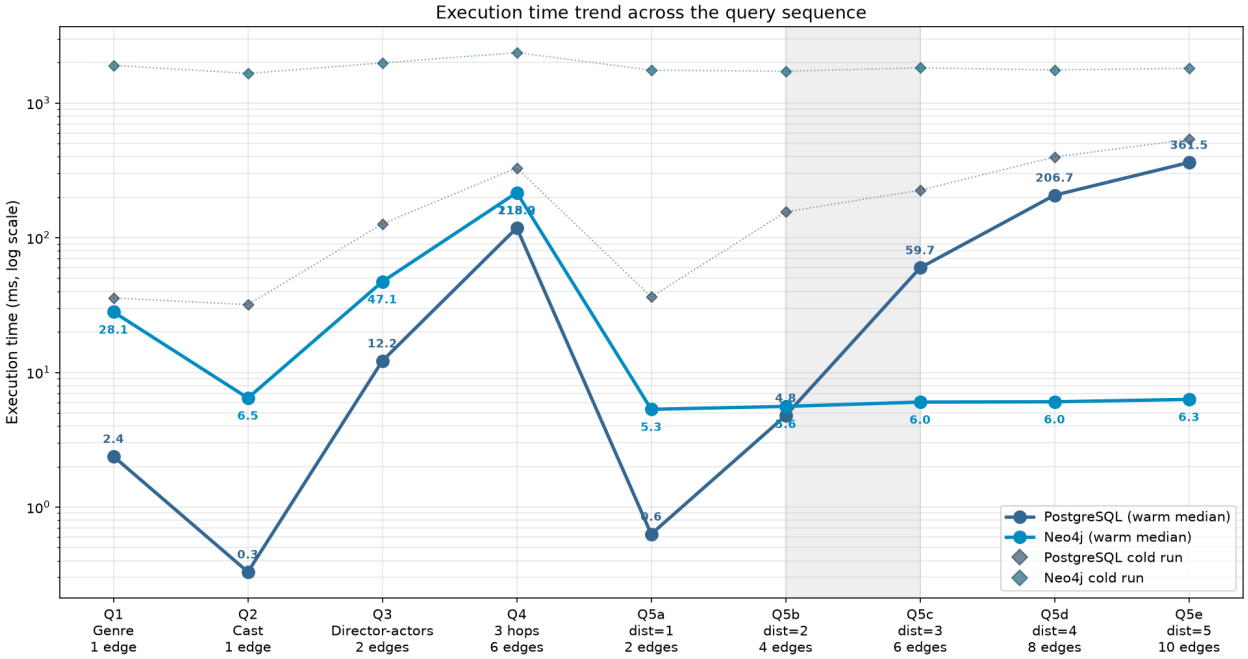


QUERY	EDGES	PG	NEO4J	FASTER
Q1	1	2.4	28.1	PostgreSQL 12×
Q2	1	0.3	6.5	PostgreSQL 22×
Q3	2	12.2	47.1	PostgreSQL 3.9×
Q4	6	118.2	215.9	PostgreSQL 1.8×
Q5a	2	0.6	5.3	PostgreSQL 8.8×
Q5b	4	4.8	5.6	PostgreSQL 1.2×
Q5c	6	59.7	6.0	Neo4j 10×
Q5d	8	206.7	6.0	Neo4j 34×
Q5e	10	361.5	6.3	Neo4j 57×

Warm median, ms · factor = slower median / faster median

The Trend

The faster system changes between Q5b and Q5c (4 → 6 edges): PostgreSQL leads up to there, Neo4j after — and its lead widens with depth.



Conclusions

On simple lookups and shallow joins **PostgreSQL** is clearly faster, up to 22 times. As queries traverse more relationships its advantage shrinks, and between four and six edges it disappears.

Beyond that point **Neo4j** takes over: its time stays around 6 ms however deep the search goes, while PostgreSQL's grows with every level, up to 57 times slower. Every SQL hop is a join; every graph hop follows a pointer.

Neither system is universally better: the choice depends on how deeply the queries traverse relationships.

Live Demo